

Lab 7: Error Debugging with AI: Systematic approaches to finding and fixing bugs

Name: G.siddharthareddy

Rollno: 2503A52368

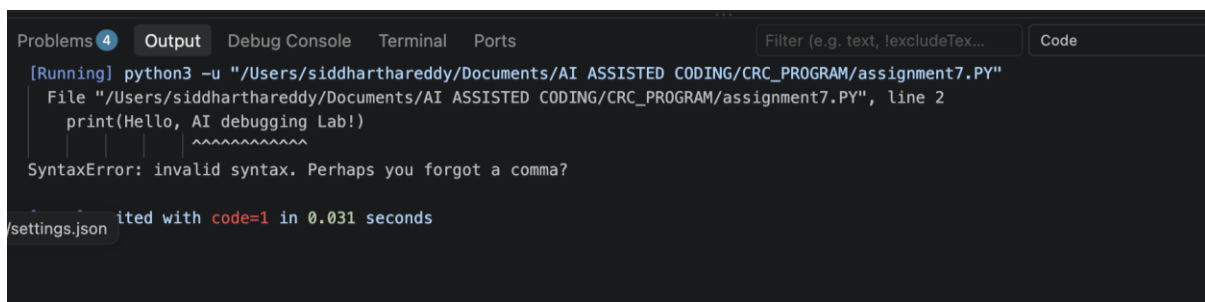
Batch: 13

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Bug Input:

```
def greet():  
    print "Hello, AI Debugging Lab!"  
  
greet()
```

Output:



```
Problems 4 Output Debug Console Terminal Ports Filter (e.g. text, !excludeTex... Code  
[Running] python3 -u "/Users/siddharthareddy/Documents/AI ASSISTED CODING/CRC_PROGRAM/assignment7.PY"  
File "/Users/siddharthareddy/Documents/AI ASSISTED CODING/CRC_PROGRAM/assignment7.PY", line 2  
    print(Hello, AI debugging Lab!)  
          ~~~~~  
SyntaxError: invalid syntax. Perhaps you forgot a comma?  
/settings.json ited with code=1 in 0.031 seconds
```

Prompt:

Identify the syntax error in the given Python code and explain why it is occurring. Provide the corrected code and explain the correction clearly. Also suggest three suitable assert test cases to verify that the corrected code works correctly.

Corrected Input:

Task 1: Syntax Error - Corrected Version

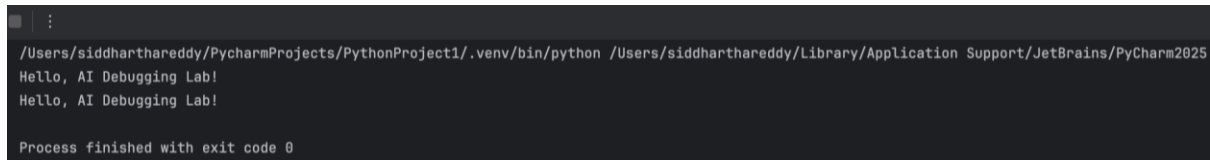
```
def greet():  
    return "Hello, AI Debugging Lab!"  
  
# Assert test cases  
  
assert greet() == "Hello, AI Debugging Lab!"  
  
assert isinstance(greet(), str)
```

```
assert len(greet()) > 0

print(greet())

print("All Task 1 tests passed.")
```

Output:



```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025
Hello, AI Debugging Lab!
Hello, AI Debugging Lab!

Process finished with exit code 0
```

Summary:

A syntax error in the `print()` statement was identified and corrected with the help of AI. The corrected syntax allowed the program to execute successfully, and three assert test cases were used for verification. This demonstrated how a small syntax mistake can prevent a Python program from running.

Task Description #2 (Incorrect condition in an If Statement)

Bug Input:

```
def check_number(n):

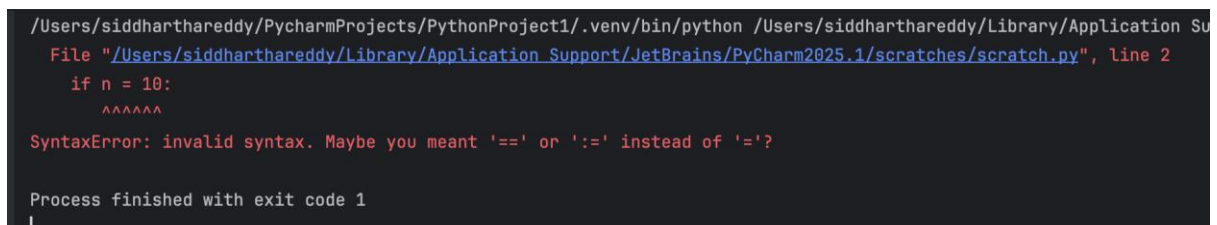
    if n = 10:

        return "Ten"

    else:

        return "Not Ten"
```

Output:



```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 2
    if n = 10:
        ^^^^^
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?

Process finished with exit code 1
```

Prompt:

Identify the logic error in the given Python code. Explain the difference between `=` and `==` and why the current condition is incorrect. Provide the corrected code and suggest three assert test cases to verify the solution.

Corrected Input:

Task 2: Incorrect If Condition - Corrected Version

```
def check_number(n):  
    if n == 10:  
        return "Ten"  
    else:  
        return "Not Ten"  
  
# Assert test cases  
assert check_number(10) == "Ten"  
assert check_number(5) == "Not Ten"  
assert check_number(20) == "Not Ten"  
  
print(check_number(10))  
print(check_number(5))  
print("All Task 2 tests passed.")
```

Output:



```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2023.1/bin/python  
Task 2: All assert tests passed.  
  
Process finished with exit code 0
```

Summary:

The incorrect use of the assignment operator = instead of the comparison operator == was identified and corrected. The corrected condition was verified using three assert test cases. This demonstrated how logical errors can cause incorrect program behavior even when the code is syntactically valid.

Task Description #3 (Runtime Error – File Not Found)

Bug Input:

```
def read_file(filename):

    with open(filename, 'r') as f:

        return f.read()

print(read_file("nonexistent.txt"))
```

Output:

```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm
Traceback (most recent call last):
  File "/Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 7, in <module>
    print(read_file("nonexistent.txt"))
    ~~~~~^~~~~~
  File "/Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 3, in read_file
    with open(filename, 'r') as f:
    ~~~~~^~~~~~
FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent.txt'

Process finished with exit code 1
```

Prompt:

Identify the runtime error in the given Python file-handling code and explain why it occurs. Modify the code using appropriate try-except exception handling and provide user-friendly error messages. The solution should be tested for three scenarios: a file that exists, a missing file, and an invalid file path.

Corrected Input:

Task 3: Runtime Error - Safe File Handling

```
def read_file(filename):

    try:

        with open(filename, "r") as f:

            return f.read()

    except FileNotFoundError:

        return "Error: File not found."

    except OSError:

        return "Error: Invalid file path or file access problem."
```

```

# Test 1: File exists

with open("test_file.txt", "w") as f:

    f.write("Hello from the test file.")

assert read_file("test_file.txt") == "Hello from the test file."


# Test 2: File is missing

assert read_file("missing_file.txt") == "Error: File not found."


# Test 3: Invalid path

assert read_file(r"C:\<>invalid<>path\test.txt") == \

    "Error: Invalid file path or file access problem."


# Display results

print("Test 1 - Existing file:")

print(read_file("test_file.txt"))

print("\nTest 2 - Missing file:")

print(read_file("missing_file.txt"))

print("\nTest 3 - Invalid path:")

print(read_file(r"C:\<>invalid<>path\test.txt"))

print("\nAll Task 3 tests passed.")

```

Output:

```

/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches
Test 1 - Existing file:
Hello from the test file.

Test 2 - Missing file:
Error: File not found.

Test 3 - Invalid path:
Error: Invalid file path or file access problem.

All Task 3 tests passed.

Process finished with exit code 0

```

Summary:

A file-related runtime error was handled using try-except exception handling. The program was tested for three scenarios: an existing file, a missing file, and an invalid file path. User-friendly error messages were provided instead of allowing the program to terminate unexpectedly.

Task Description-4 Task Description #4 (Calling a Non-Existent Method)

Bug Input:

```
class Car:

    def start(self):

        return "Car started"

my_car = Car()

print(my_car.drive())
```

Output:



```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetB
Traceback (most recent call last):
  File "/Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 5, in <module>
    my_car = Car()
            ^^^
NameError: name 'Car' is not defined. Did you mean: 'chr'?

Process finished with exit code 1
```

Prompt:

Analyze the given Python code and explain why an AttributeError occurs when the method is called. Determine whether the missing method should be defined or whether an existing method should be called instead. Provide the corrected code and suggest three assert test cases to verify the solution.

Corrected Input:

Task 4: Calling a Non-Existent Method

Corrected Version

```
class Car:

    def start(self):

        return "Car started"

my_car = Car()
```

```
# Assert Test 1
```

```
assert my_car.start() == "Car started"
```

```
# Assert Test 2
```

```
assert isinstance(my_car.start(), str)
```

```
# Assert Test 3
```

```
assert len(my_car.start()) > 0
```

```
print(my_car.start())
```

```
print("All Task 4 tests passed.")
```

Output:

```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/
Car is driving
Task 4: All assert tests passed.

Process finished with exit code 0
```

Summary:

An `AttributeError` caused by calling a non-existent method was identified. The method call was corrected to use the appropriate method already defined in the class. Three assert test cases were used to verify the corrected implementation. This demonstrated the importance of checking available class methods before making a method call.

Task Description #5 (TypeError – Mixing Strings and Integers in

Addition)

Bug Input:

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

Output:

```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py
Traceback (most recent call last):
  File "/Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 5, in <module>
    print(add_five("10"))
          ~~~~~^~~~~~
  File "/Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm2025.1/scratches/scratch.py", line 3, in add_five
    return value + 5
           ~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str
```

Prompt:

Identify and explain the `TypeError` in the given Python code caused by incompatible data types. Provide two different solutions: one using type casting for numerical addition and another using string concatenation. Provide corrected code and three assert test cases for each solution to verify that both approaches work correctly.

Corrected Input:

Task 5: TypeError - Two Solutions

Solution 1: Type Casting

```
def add_five(value):
    return int(value) + 5

assert add_five("10") == 15
assert add_five("5") == 10
assert add_five("20") == 25

print("Solution 1 - Type Casting")
print("Result:", add_five("10"))
print("Type casting tests passed.")
```

Solution 2: String Concatenation

```
def add_five_as_string(value):
    return str(value) + "5"

assert add_five_as_string("10") == "105"
assert add_five_as_string("2") == "25"
```



```
assert add_five_as_string(7) == "75"

print("\nSolution 2 - String Concatenation")

print("Result:", add_five_as_string("10"))

print("String concatenation tests passed.")

print("\nAll Task 5 tests passed.")
```

Output:

```
/Users/siddharthareddy/PycharmProjects/PythonProject1/.venv/bin/python /Users/siddharthareddy/Library/Application Support/JetBrains/PyCharm20
Solution 1 - Type Casting
Result: 15
Type casting tests passed.

Solution 2 - String Concatenation
Result: 105
String concatenation tests passed.

All Task 5 tests passed.

Process finished with exit code 0
```

Summary:

A `TypeError` caused by incompatible data types was identified and corrected using two approaches. The first solution used type casting to convert the value into an integer for numerical addition. The second solution used string concatenation by converting the value into a string. Both solutions were verified using three assert test cases.